PARTE 2 DE 4 · 3 MINUTOS

Alex Monroy

Modelo de datos: entidades, relaciones y repositorios

Explicas **cómo se representan los datos**: las tres entidades, cómo se relacionan, el enum de estados y cómo los repositorios consultan la base de datos sin escribir SQL. Lo que más se evalúa aquí son las relaciones.

Orden de la exposición

#	Quién	Tema	Tiempo
1	Santiago Arango	Introducción, tecnologías y arquitectura	3 min
2	Alex Monroy	Modelo de datos: entidades, relaciones y repositorios	3 min
3	Jose Vasquez	Lógica de negocio: servicios, reglas y excepciones	3-4 min
4	Isaac	API REST, validaciones, errores y demo en vivo	4-5 min

Antes de empezar

- Abiertos: **Student.java**, **Enrollment.java**, **EnrollmentStatus.java** y **EnrollmentRepository.java**.
- phpMyAdmin abierto en **mi_base_de_datos** (opcional).

Guion paso a paso

Cada paso dice qué mostrar en pantalla y qué decir. No lo leas palabra por palabra: úsalo como guía y dilo con tus palabras.

1. El modelo

```
Student  1 ——— N  Enrollment  N ——— 1  Course
```

QUÉ DECIR

Tenemos tres entidades: Student, Course y Enrollment, que es la matrícula. Un estudiante puede estar en varios cursos y un curso tiene varios estudiantes, así que es muchos a muchos; pero lo resolvimos con una entidad intermedia, porque la matrícula tiene datos propios: la fecha y el estado.

2. Entidades Student y Course

Muestra: Student.java

```
@Entity
@Table(name = "students")
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false)
    private String email;

    @OneToMany(mappedBy = "student", cascade = CascadeType.ALL)
    private List<Enrollment> enrollments;
}
```

QUÉ DECIR

Con @Entity esta clase se vuelve una tabla y el id es autoincremental. El correo es unique, así que no se puede repetir. La lista de matrículas es @OneToMany con mappedBy, y el cascade ALL hace que al borrar un estudiante se borren sus matrículas. Course es igual, con código, nombre y un cupo máximo.

3. Entidad Enrollment y su estado

Muestra: Enrollment.java y EnrollmentStatus.java

```
@ManyToOne
@JoinColumn(name = "student_id", nullable = false)
private Student student;

@ManyToOne
@JoinColumn(name = "course_id", nullable = false)
private Course course;

@Enumerated(EnumType.STRING)
private EnrollmentStatus status;   // ACTIVE, CANCELLED, COMPLETE
```

QUÉ DECIR

La matrícula tiene dos @ManyToOne, uno al estudiante y otro al curso. @JoinColumn crea las llaves foráneas, y nullable false impide una matrícula sin estudiante o sin curso. El estado es un enum con tres valores, y lo guardamos como texto para que los datos no se dañen si cambia el orden del enum.

4. Repositorios

Muestra: EnrollmentRepository.java (y si alcanzas, las tablas en phpMyAdmin).

```
public interface EnrollmentRepository extends JpaRepository<Enrollment, Long> {
    long countByCourse_IdAndStatus(Long courseId, EnrollmentStatus status);
}
```

QUÉ DECIR

Para la base de datos usamos repositorios que extienden JpaRepository, y con eso ya tenemos guardar, listar, buscar y eliminar. Además Spring arma la consulta a partir del nombre del método: este cuenta cuántas matrículas tiene un curso en cierto estado. Jose las va a usar para las reglas de negocio.

Preguntas que te pueden hacer

¿Por qué no usaron @ManyToMany?

Porque solo crea una tabla con los dos ids, y necesitábamos guardar también la fecha y el estado.

¿Qué significa mappedBy?

Que el dueño de la relación es Enrollment, que es el que tiene la llave foránea.

¿Qué pasa si borro un curso con matrículas?

Por el cascade ALL se borran también sus matrículas, y no quedan registros huérfanos.

¿Qué SQL genera countByCourse_IdAndStatus?

SELECT COUNT(*) FROM enrollment WHERE course_id = ? AND status = ?

Cierre y paso al siguiente

QUÉ DECIR

Ya vimos cómo se guardan los datos. Ahora Jose les muestra las reglas que se revisan antes de guardar.

Consejos

- Si phpMyAdmin no carga, no pierdas tiempo: sigue con los repositorios.
- Ensayen una vez los cuatro seguidos con cronómetro. Si alguien se pasa, el que sufre es la demo del final.
- Cuando muestren código, hagan zoom en IntelliJ (Ctrl + rueda del mouse) para que se lea desde atrás.
- Si no saben una respuesta, digan qué parte del código la resuelve o que lo revisan; es mejor que inventar.